

Tutorial 1

Introducción a Scratch

Introducción

En este tutorial tendrás oportunidad de ejercitar tus conocimientos sobre Scratch y el pensamiento computacional, trabajando en una serie de problemas de dificultad creciente. Es muy importante que aproveches de hacer preguntas en el [foro](#) y revises los vídeos publicados en Canvas, para aclarar tus dudas y avanzar en cada uno de los problemas. Para comenzar, revisaremos algunos conceptos importantes que debes comprender para programar con Scratch.

Revisión de Conceptos

1. Describe en palabras cómo puedes obtener un dato del usuario y guardarlo en una variable en Scratch.
¿Qué bloques debes usar para lograr esto?
R: Se puede usar un bloque de tipo **ask** que guarda el valor ingresado en la variable **answer**. Luego, suponiendo que existe alguna variable creada en el programa, se puede utilizar el bloque **set** para asignar el valor de **answer** a dicha variable.
2. ¿Cómo puedes aumentar o disminuir el valor de una variable numérica en Scratch?
R: Se puede utilizar el bloque **change-by**, pasando a **change** la variable a cambiar, y en **by** un valor positivo para aumentar, o negativo para disminuir.
3. ¿Cómo puedes desplegar en Scratch un mensaje que contenga texto y el valor almacenado en una variable?
R: Se puede utilizar el bloque **join** para unir el texto y el contenido de la variable. El resultado de la operación **join** se puede desplegar con un bloque **say**.
4. Si en un programa en Scratch hay un bloque del tipo **if-else**, ¿puede darse un caso en que se ejecuten los códigos de ambas ramas?
R: En ningún caso pueden ejecutarse ambas ramas, pues son mutuamente excluyentes porque la condición del bloque **if** no puede ser verdadera y falsa a la vez.
5. ¿Cuál es la diferencia entre **if** y **repeat** en Scratch?
R: **if** sirve para decidir un camino de ejecución condicionado al valor de verdad (verdadero o falso) en una expresión lógica. En cambio **repeat** permite que un conjunto de bloques de Scratch pueda ejecutarse en forma repetitiva, hasta que cierta condición se cumpla.

Problema 1

Construye un programa que reciba la nota final de un alumno del 1 al 7 y despliegue un mensaje de acuerdo a las siguientes reglas:

1. Si la nota obtenida es mayor o igual a 6,0 y menor o igual a 7,0 → El mensaje debe ser “¡Eximido!”.
2. Si la nota obtenida está entre 4,0 y 5,9, incluyendo ambos extremos → El mensaje debe ser “Aprobado”.
3. Si la nota obtenida es menor o igual a 3,9 y mayor o igual a 1,0 → El mensaje debe ser “Reprobado”.
4. En otro caso → El mensaje debe ser “No entiendo la nota ingresada”.

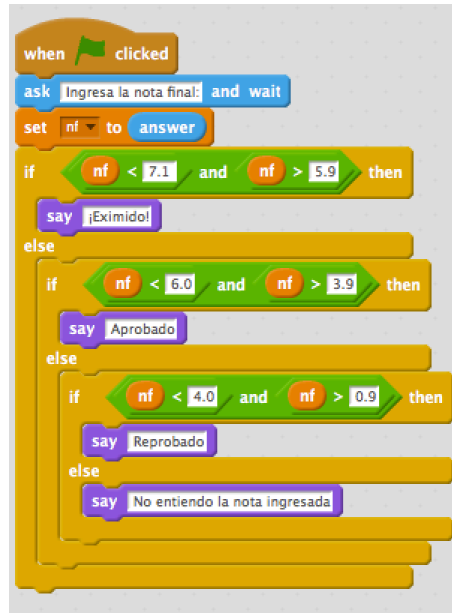


Figura 1: Solución del problema 1

La clave en esta solución es entender cómo se construyen expresiones lógicas para ir verificando si la variable que contiene la nota final cae dentro de los rangos de acuerdo a las reglas. Usamos el conectivo lógico **and** para ver si el valor en la variable **nf** satisface la condición al lado izquierdo y al lado derecho a la vez. Si alguna de las dos condiciones no se cumple – o ninguna de ellas – la ejecución del programa continúa en el bloque bajo **else** en cada caso.

Problema 2

Construye un programa que reciba una palabra del usuario, y luego decida el tipo de respuesta a desplegar, considerando las siguientes reglas:

1. Si el largo de la palabra es menor a 3 caracteres → La respuesta debe ser “Yo no entiendo monosílabos”.
2. Si el largo de la palabra es 3 ó más caracteres, pero menos de 9, → La respuesta debe ser “Sí, y qué?”.
3. Si el largo de la palabra es de 9 o más caracteres → La respuesta debe ser “Yo no entiendo palabras difíciles”.

Para conocer el largo de la palabra ingresada por el usuario, puedes utilizar el bloque **operators/length of**.

Puedes probar tu programa libremente con palabras que ojalá sean respetuosas.

Solución



Figura 2: Solución del problema 2

La lógica de la solución a este problema es similar al problema anterior, consiste en establecer un pequeño “árbol de decisión” en donde el largo de la palabra es comparado con las distintas posibilidades. En el segundo bloque `if` ya se sabe que el largo de la palabra es mayor o igual a 3. Finalmente, si el largo de la palabra no es menor a 9, entonces la palabra es de 9 o más caracteres, y se ejecuta el último bloque `else`.

Problema 3

Construye un programa que pida un número entero y muestre todos los divisores entre 2 y 9 de dicho número. Los divisores pueden ir apareciendo uno a uno, con *sleep* de un segundo entremedio.

Para verificar la divisibilidad en Scratch, se utiliza el bloque `operators/mod` (color verde). Este bloque recibe dos valores en sus lados izquierdo y derecho. Si el valor en el lado derecho divide al valor del lado izquierdo, el resultado de `mod` es cero. En caso contrario, el valor es distinto de cero. Puedes ver un [vídeo explicativo aquí](#).

Solución

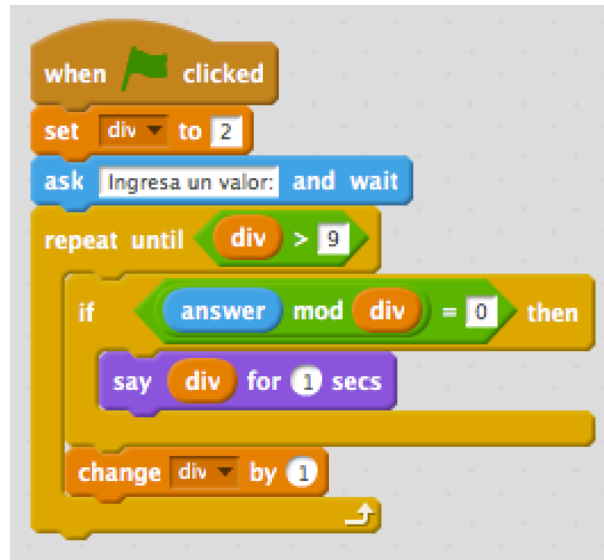


Figura 3: Solución del problema 3

La idea de la solución es ir generando cada uno de los divisores de 2 a 9 utilizando la iteración del bloque **repeat**. La variable **div** que contiene el valor del divisor actual, va aumentando hasta superar 9, y en cada iteración se verifica si el valor en **div** divide al valor ingresado por el usuario. Para verificar la divisibilidad, se utiliza el bloque **a mod b** que devuelve 0 si efectivamente el valor **a** a la derecha divide al valor de la izquierda.

Problema 4

FizzBuzz es un juego en que se deben decir los números naturales, del 1 en adelante, pero los números que son divisibles por 3 se reemplazan por “Fizz”, los divisibles por 5 por “Buzz”, y los que son divisibles por 3 y por 5 se reemplazan por “FizzBuzz”. Escribe un programa que despliegue el resultado de jugar FizzBuzz contando hasta 100, con sleep de un segundo entre números sucesivos. Para esto hay que usar el bloque **operators/mod** explicado en el problema anterior.

Ejemplo de salida hasta el 18:

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
```

FizzBuzz

16

17

Fizz

Solución

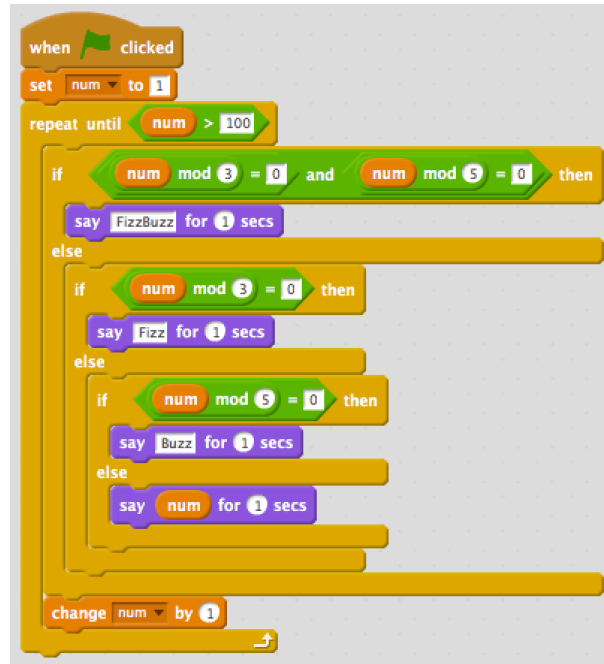


Figura 4: Solución del problema 4

La solución consiste en una iteración que va incrementando el valor actual. Se utiliza un árbol de decisión para saber si el número cumple con las condiciones para FizzBuzz, que deben verificarse antes de la condición de Fizz y la condición de Buzz. De lo contrario, podríamos caer en el error de imprimir Fizz y FizzBuzz en caso que el número sea divisible por 3 y 5, o Buzz FizzBuzz.

Problema 5

Construye un programa que genere un número al azar entre 1 y 100 y haga que el usuario tenga que adivinarlo. Si el número ingresado por el usuario es menor o mayor, el programa debe decirle “el número ingresado es mayor (o menor) al buscado”. Implementa el programa en tres etapas sucesivas:

1. El usuario tiene un solo intento para adivinar.
2. El usuario tiene hasta tres intentos para adivinar.
3. El usuario puede indicar al inicio cuántos intentos quiere para adivinar.

Solución

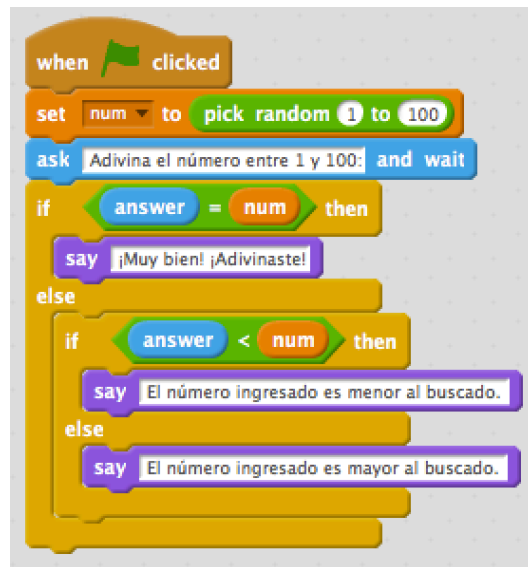


Figura 5: Solución del problema 5-1

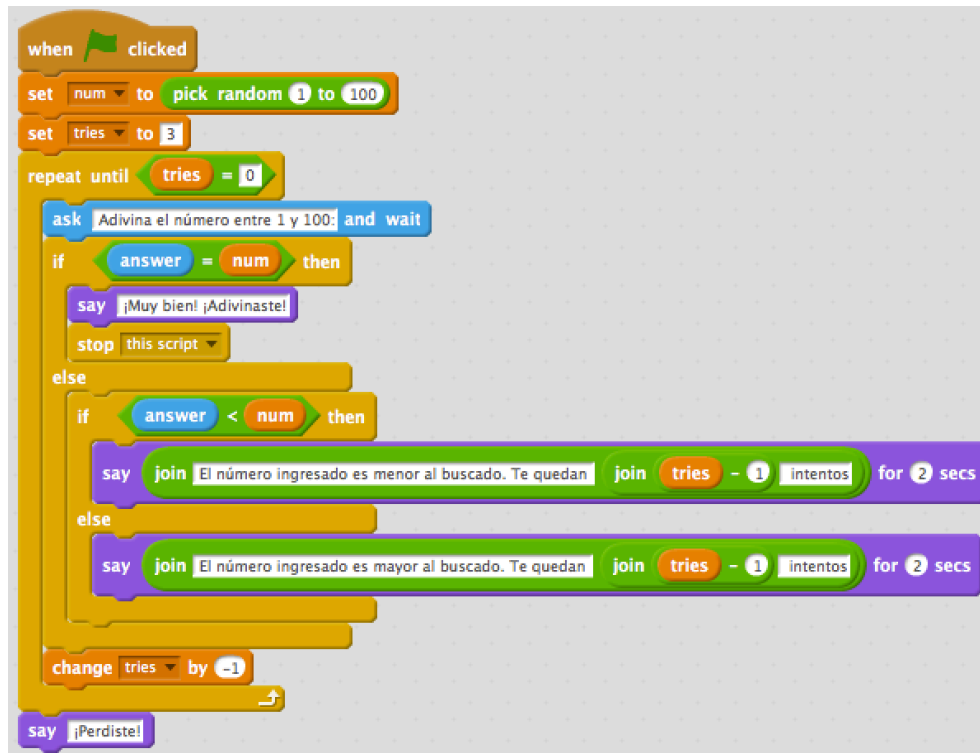


Figura 6: Solución del problema 5-2

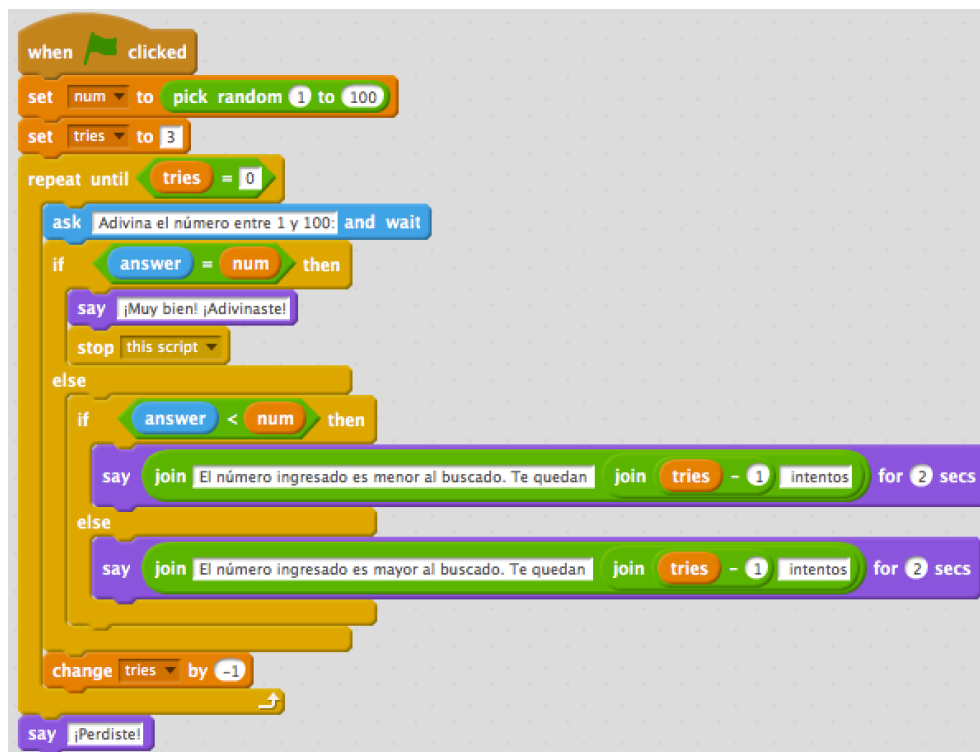


Figura 7: Solución del problema 5-3

La solución a la primera parte es sencilla. En la segunda parte se debe incorporar una iteración que lleve la cuenta de los intentos. En la última parte, se utiliza un valor ingresado por el usuario para controlar la cantidad de iteraciones.